Urban Traffic Simulator

1. Project Overview

In modern cities, transportation systems must continuously respond to changing traffic conditions such as congestion, accidents, and increasing vehicle density. Navigation systems rely on graph algorithms and real-time simulation to compute efficient travel routes and optimize traffic flow.

In this project, students will develop an **Urban Traffic Simulator**, an object-oriented software application that simulates traffic movement within a virtual city environment. The simulator models:

- roads,
- intersections,
- vehicles,
- traffic conditions,
- routing strategies,

as interacting software objects.

The system acts as a simplified **urban digital twin**, where a virtual traffic network mirrors real-world traffic behavior and dynamically adapts to changing conditions.

The project integrates:

- object-oriented programming,
- graph data structures,
- pathfinding algorithms,
- simulation systems,
- event-driven architecture,
- software engineering practices,
- collaborative development using Git.

The final product must be a fully runnable application capable of simulating traffic behavior and evaluating routing algorithms under multiple scenarios.

2. Team Requirements

- Each team consists of **4 students**.
- All members are expected to contribute technically.
- Every member must:
    - write code,
    - commit to Git regularly,
    - participate in system design,
    - contribute during debugging and testing,
    - present during the oral presentation.

Contribution imbalance may affect individual grading.

3. Development Duration

Total project duration: 8 weeks

Students are expected to continuously develop the system throughout the semester rather than concentrating work near the deadline.

4. Functional Requirements

The project must implement the following functionalities.

4.1 Road Network System

The traffic environment must be represented as a graph-based road network.

Required Features

Intersections

Each intersection should:

- have a unique identifier,
- maintain connected roads,
- support incoming/outgoing traffic.

Roads

Each road should contain:

- start intersection,
- destination intersection,
- distance,
- speed limit,
- congestion level,
- travel cost.

Graph Operations

The system must support:

- adding/removing roads,
- adding/removing intersections,
- updating road conditions,
- searching connected roads.

Map Loading

The simulator should support:

- loading road networks from file,
- or generating synthetic maps automatically.

Example file formats:

- CSV,
- JSON,
- text configuration files.

4.2 Vehicle Simulation System

Vehicles are active objects moving inside the traffic network.

Vehicle Properties

Each vehicle should maintain:

- unique ID,
- current position,
- destination,
- current route,
- movement speed,
- travel history.

Vehicle Behaviors

Vehicles should be able to:

- request routes,
- move step-by-step,
- react to congestion,
- recompute routes dynamically,
- stop when the destination is reached.

Multiple Vehicle Support

The simulator must support:

- multiple simultaneous vehicles,
- independent routing decisions,
- congestion caused by vehicle density.

4.3 Routing and Navigation System

The core functionality of the project is dynamic route planning.

Students must implement at least:

- one baseline routing algorithm,
- one advanced routing algorithm.

Required Algorithms

Breadth-First Search (BFS)

Used as a simple baseline for unweighted traversal.

Dijkstra Algorithm

Computes shortest weighted paths.

A* Search

Uses heuristic estimation to improve search efficiency.

Route Planning Features

The routing system should support:

- shortest-distance routing,
- shortest-time routing,
- dynamic path recomputation,
- congestion-aware navigation.

Dynamic Routing

Vehicles should update routes when:

- traffic congestion changes,
- roads become blocked,
- accidents occur.

This is one of the most important functionalities of the project.

4.4 Traffic Event System

The simulator must support dynamic traffic events.

Possible Events

Congestion Event

Traffic density increases travel cost.

Accident Event

Road capacity decreases or becomes unavailable.

Road Closure

The road is temporarily disabled.

Traffic Light Delay

Vehicles wait at intersections.

Event Management

The system should:

- trigger events during simulation,
- notify affected vehicles,
- update routing conditions dynamically.

Students are encouraged to implement event-driven architecture.

4.5 Simulation Engine

The simulation engine coordinates all system activities.

Responsibilities

- updating simulation state,
- advancing simulation time,
- processing vehicle movement,

- handling traffic events,
- collecting statistics.

Simulation Features

Time Step Simulation

The system updates periodically using simulation ticks.

Pause/Resume

Simulation should support pausing and restarting.

Speed Control

- (Optional feature) adjustable simulation speed, often known as fast time simulation.

4.6 Visualization System

The simulator should visually demonstrate traffic behavior.

Minimum Visualization Requirements

The system must display:

- intersections,
- roads,
- vehicle movement,
- route paths,
- congestion indicators.

Suggested GUI Features

Road Visualization

Display roads as connected graph edges.

Vehicle Animation

Vehicles move dynamically on the map.

Congestion Heatmap

Road color changes based on congestion level.

Statistics Panel

Display:

- average travel time,
- number of vehicles,
- congestion metrics.

4.7 Statistics and Analysis Module

The system should analyze traffic performance.

Metrics

Travel Metrics

- total travel time,
- average travel time,
- route length.

Algorithm Metrics

- routing computation time,
- number of route recalculations,
- path optimality.

Traffic Metrics

- congestion level,
- vehicle density,

- blocked roads.

Experimentation

Students should compare algorithms under:

- normal traffic,
- heavy congestion,
- random events,
- large-scale networks.

Results should be summarized in charts or tables.

5. Object-Oriented Programming Requirements

This is an OOP-focused project.

Students must demonstrate proper usage of:

Encapsulation

Objects manage their own internal state.

Inheritance

Routing algorithms inherit from abstract interfaces.

Polymorphism

Algorithms can be switched dynamically.

Abstraction

Simulation components interact through abstract APIs.

6. Required Software Engineering Practices

Students are expected to apply professional software engineering practices.

Modular Design

The project should be separated into:

- simulation layer,
- algorithm layer,
- visualization layer,
- utility layer.

Code Readability

Code should:

- follow naming conventions,
- contain meaningful comments,
- avoid duplicated logic.

Error Handling

The system should handle:

- invalid map files,
- disconnected roads,
- invalid destinations,
- empty graphs.

Testing

Students are encouraged to implement:

- unit tests,
- edge-case testing,
- stress testing.

7. (Suggested) System Architecture

Core Classes

Graph Components

- `Graph`
- `Road`
- `Intersection`

Vehicle Components

- `Vehicle`
- `EmergencyVehicle`
- `Bus`
- `Car`
- `(Optional) Bike/motorbike, pedestrian`

Algorithm Components

- `PathFindingStrategy`
- `DijkstraStrategy`
- `AStarStrategy`

Simulation Components

- `TrafficSimulator`
- `TrafficEvent`
- `TrafficController`

Utility Components

- `StatisticsManager`
- `MapLoader`
- `VisualizationEngine`

8. Design Patterns

Students should apply at least two design patterns.

Recommended patterns:

Pattern	Purpose
Strategy Pattern	Routing algorithms
Observer Pattern	Traffic updates
Factory Pattern	Vehicle generation
Singleton Pattern	Simulation manager
MVC	GUI separation

9. Git and Collaboration Requirements

Mandatory Git Usage

Each team must maintain a Git repository throughout the project.

Weekly Progress Tracking

The instructor will monitor via git commits:

- commit frequency,
- development progress,
- contribution balance,
- repository activity.

Contribution Requirements

Every member must:

- commit regularly,
- contribute meaningful code,

- participate throughout the semester.

Minimal or inactive contribution may reduce individual scores.

Recommended Workflow

Students are encouraged to use:

- feature branches,
- pull requests,
- issue tracking,
- descriptive commit messages.

Private Repository

Each team's source code repository must be accessible only to team members. External sharing is not allowed.

Documentation

Maintain a brief summary of the source code and a tutorial on how to run the project in the `README.md` file located in the root directory of the repository.

10. (Suggested) Development Timeline

Week 2

- Requirement analysis
- Team planning
- System architecture
- Repository setup

Week 3

- Graph implementation
- Map loading system

Week 4

- Routing algorithms
- Basic vehicle movement

Week 5

- Dynamic traffic events
- Simulation engine

Week 6

- Visualization system
- Statistics collection

Week 7

- Optimization
- Testing and debugging

Week 8

- Final polishing
- Documentation
- Oral presentation

11. Deliverables

Each team must submit:

Source Code

- fully runnable project,
- complete Git repository,
- setup instructions.

Documentation

- architecture description,
- class diagram,
- algorithm explanation,
- experiment summary.

Dataset / Configuration

- road network files,
- simulation configuration.

Oral Presentation

Each team will conduct a live presentation and demonstration.

Presentation Requirements

- every member must present,
- explain architecture and implementation,
- demonstrate the simulator,
- discuss experiments and challenges,
- answer technical questions from instructor(s).

Students must demonstrate understanding of their own code during Q&A.